Unit –III

Mining Frequent Patterns : Basic Concepts, Apriori Algorithm: Finding Frequent Item sets by Confined Candidate Generation, Generating Association Rules from Frequent Item sets, Mining Multilevel Associations, Constraint-Based Frequent Pattern Mining.

Finding recurrent patterns or item sets in huge datasets is the goal of frequent pattern mining, a crucial data mining approach. It looks for groups of objects that regularly appear together in order to expose underlying relationships and interdependence. Market basket analysis, web usage mining, and bioinformatics are a few areas where this method is important.

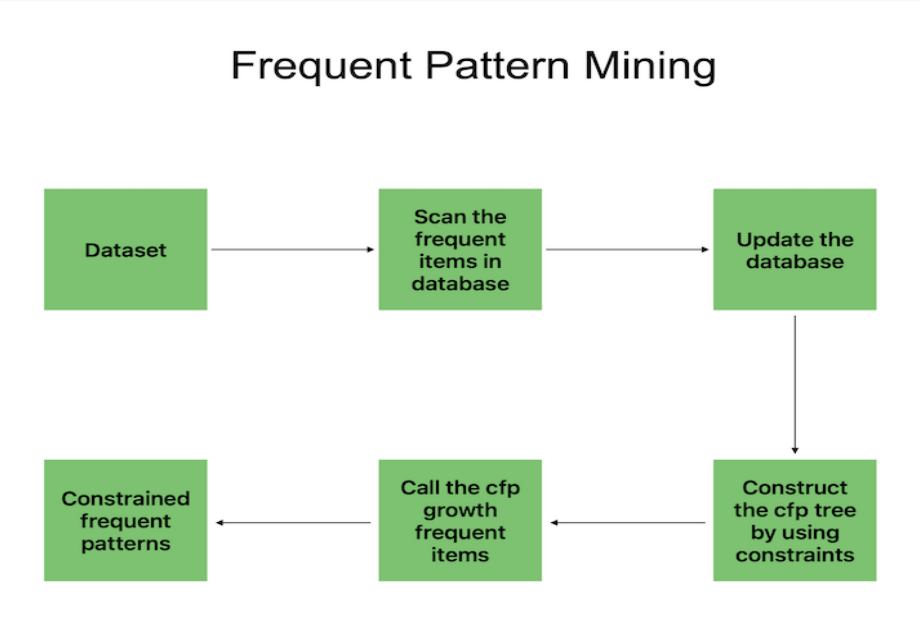
It helps organizations comprehend client preferences, optimize cross−selling tactics, and improve recommendation systems by revealing patterns of consumer behavior. By examining user navigational habits and customizing the browsing experience, online use mining aids in enhancing website performance.

Basic Concepts in Frequent Pattern Mining

The technique of frequent pattern mining is built upon a number of fundamental ideas. The analysis is based on transaction databases, which include records or transactions that represent collections of objects. Items inside these transactions are grouped together as itemsets.

The importance of patterns is greatly influenced by support and confidence measurements. Support quantifies how frequently an itemset appears in the database, whereas confidence quantifies how likely it is that a rule generated from the itemset is accurate.

The Apriori algorithm, a popular method for finding recurrent patterns, takes a methodical approach. In order to find no more frequent itemsets, it generates candidate itemsets, prunes the infrequent ones, and then progressively grows the size of the itemsets. The patterns that fulfill the required support criteria are successfully identified through this iterative approach.



Techniques for Frequent Pattern Mining

Apriori Algorithm

One of the most popular methods, the Apriori algorithm, uses a step−by−step procedure to find frequent item sets. It starts by creating candidate itemsets of length 1, determining their support, and

eliminating any that fall below the predetermined cutoff. The method then joins the frequent itemsets from the previous phase to produce bigger itemsets repeatedly.

Once no more common item sets can be located, the procedure is repeated. The Apriori approach is commonly used because of its efficiency and simplicity, but because it requires numerous database scans for big datasets, it can be computationally inefficient.

Apriori algorithm is given by R. Agrawal and R. Srikant in 1994 for finding frequent itemsets in a dataset for boolean association rule. Name of the algorithm is Apriori because it uses prior knowledge of frequent itemset properties. We apply an iterative approach or level-wise search where k-frequent itemsets are used to find k+1 itemsets.

To improve the efficiency of level-wise generation of frequent itemsets, an important property is used called *Apriori property* which helps by reducing the search space.

Apriori Property –
All non-empty subset of frequent itemset must be frequent. The key concept of Apriori algorithm is its anti-monotonicity of support measure. Apriori assumes that

All subsets of a frequent itemset must be frequent(Apriori property).
If an itemset is infrequent, all its supersets will be infrequent.

Before we start understanding the algorithm, go through some definitions which are explained in my previous post.
Consider the following dataset and we will find frequent itemsets and generate association rules for them.

TID	items
T1	I1, I2 , I5
T2	I2,I4
T3	I2,I3
T4	I1,I2,I4
T5	I1,I3
T6	I2,I3
T7	I1,I3
T8	I1,I2,I3,I5
T9	I1,I2,I3

minimum support count is 2
minimum confidence is 60%

Step-1: K=1
(I) Create a table containing support count of each item present in dataset – Called **C1(candidate set)**

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

(II) compare candidate set item's support count with minimum support count(here min_support=2 if support_count of candidate set items is less than min_support then remove those items). This gives us itemset L1.

This gives us itemset L1.

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

Step-2: K=2

- Generate candidate set C2 using L1 (this is called join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common.
- Check all subsets of an itemset are frequent or not and if not frequent remove that itemset.(Example subset of{I1, I2} are {I1}, {I2} they are frequent.Check for each itemset)
- Now find support count of these itemsets by searching in dataset.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I4	1
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I3,I4	0
I3,I5	1
I4,I5	0

(II) compare candidate (C2) support count with minimum support count(here min_support=2 if support_count of candidate set item is less than min_support then remove those items) this gives us itemset L2.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I2,I5	2

Step-3:

- Generate candidate set C3 using L2 (join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common. So here, for L2, first element should match.
  So itemset generated by joining L2 is {I1, I2, I3}{I1, I2, I5}{I1, I3, i5}{I2, I3, I4}{I2, I4, I5}{I2, I3, I5}
- Check if all subsets of these itemsets are frequent or not and if not, then remove that itemset.(Here subset of {I1, I2, I3} are {I1, I2},{I2, I3},{I1, I3} which are frequent. For {I2, I3, I4}, subset {I3, I4} is not frequent so remove it. Similarly check for every itemset)
- find support count of these remaining itemset by searching in dataset.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

(II) Compare candidate (C3) support count with minimum support count(here min_support=2 if support_count of candidate set item is less than min_support then remove those items) this gives us itemset L3.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

Step-4:

- Generate candidate set C4 using L3 (join step). Condition of joining L_{k-1} and L_{k-1} (K=4) is that, they should have (K-2) elements in common. So here, for L3, first 2 elements (items) should match.
- Check all subsets of these itemsets are frequent or not (Here itemset formed by joining L3 is {I1, I2, I3, I5} so its subset contains {I1, I3, I5}, which is not frequent). So no itemset in C4
- We stop here because no frequent itemsets are found further

---

Thus, we have discovered all the frequent item-sets. Now generation of strong association rule comes into picture. For that we need to calculate confidence of each rule.

Confidence –
A confidence of 60% means that 60% of the customers, who purchased milk and bread also bought butter.

```
Confidence(A->B)=Support_count(AUB)/Support_count(A)
```

So here, by taking an example of any frequent itemset, we will show the rule generation.
Itemset {I1, I2, I3} //from L3
SO rules can be
[I1^I2]=>[I3] //confidence = sup(I1^I2^I3)/sup(I1^I2) = 2/4*100=50%
[I1^I3]=>[I2] //confidence = sup(I1^I2^I3)/sup(I1^I3) = 2/4*100=50%
[I2^I3]=>[I1] //confidence = sup(I1^I2^I3)/sup(I2^I3) = 2/4*100=50%
[I1]=>[I2^I3] //confidence = sup(I1^I2^I3)/sup(I1) = 2/6*100=33%

[I2]=>[I1^I3] //confidence = sup(I1^I2^I3)/sup(I2) = 2/7*100=28%
[I3]=>[I1^I2] //confidence = sup(I1^I2^I3)/sup(I3) = 2/6*100=33%

So if minimum confidence is 50%, then first 3 rules can be considered as strong association rules

Frequent Item set in Data set (Association Rule Mining)

INTRODUCTION:

1. Frequent item sets, also known as association rules, are a fundamental concept in association rule mining, which is a technique used in data mining to discover relationships between items in a dataset. The goal of association rule mining is to identify relationships between items in a dataset that occur frequently together.
2. A frequent item set is a set of items that occur together frequently in a dataset. The frequency of an item set is measured by the support count, which is the number of transactions or records in the dataset that contain the item set. For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20.
3. Association rule mining algorithms, such as Apriori or FP-Growth, are used to find frequent item sets and generate association rules. These algorithms work by iteratively generating candidate item sets and pruning those that do not meet the minimum support threshold. Once the frequent item sets are found, association rules can be generated by using the concept of confidence, which is the ratio of the number of transactions that contain the item set and the number of transactions that contain the antecedent (left-hand side) of the rule.
4. Frequent item sets and association rules can be used for a variety of tasks such as market basket analysis, cross-selling and recommendation systems. However, it should be noted that association rule mining can generate a large number of rules, many of which may be irrelevant or uninteresting. Therefore, it is important to use appropriate measures such as lift and conviction to evaluate the interestingness of the generated rules.

Association Mining searches for frequent items in the data set. In frequent mining usually, interesting associations and correlations between item sets in transactional and relational databases are found. In short, Frequent Mining shows which items appear together in a transaction or relationship.

Need of Association Mining: Frequent mining is the generation of association rules from a Transactional Dataset. If there are 2 items X and Y purchased frequently then it's good to put them together in stores or provide some discount offer on one item on purchase of another item. This can really increase sales. For example, it is likely to find that if a customer buys **Milk** and **bread** he/she also buys **Butter**. So the association rule is **['milk]^['bread']=>['butter']**. So the seller can suggest the customer buy butter if he/she buys Milk and Bread.

Important Definitions :

- **Support :** It is one of the measures of interestingness. This tells about the usefulness and certainty of rules. **5% Support** means total 5% of transactions in the database follow the rule.

```
Support(A -> B) = Support_count(A ∪ B)
```

- **Confidence:** A confidence of 60% means that 60% of the customers who purchased a milk and bread also bought butter.

```
Confidence(A -> B) = Support_count(A ∪ B) / Support_count(A)
```

If a rule satisfies both minimum support and minimum confidence, it is a strong rule.

- **Support_count(X)**: Number of transactions in which X appears. If X is A **union** B then it is the number of transactions in which A and B both are present.

- **Maximal Itemset:** An itemset is maximal frequent if none of its supersets are frequent.
- **Closed Itemset:** An itemset is closed if none of its immediate supersets have same support count same as Itemset.
- **K- Itemset:** Itemset which contains K items is a K-itemset. So it can be said that an itemset is frequent if the corresponding support count is greater than the minimum support count.

Example On finding Frequent Itemsets – Consider the given dataset with given transactions.

TransactionId	Items
1	{A,C,D}
2	{B,C,D}
3	{A,B,C,D}
4	{B,D}
5	{A,B,C,D}

- Lets say minimum support count is 3
- Relation hold is maximal frequent => closed => frequent

1-frequent: {A} = 3; // not closed due to {A, C} and not maximal {B} = 4; // not closed due to {B, D} and no maximal {C} = 4; // not closed due to {C, D} not maximal {D} = 5; // closed item-set since not immediate super-set has same count. Not maximal

2-frequent: {A, B} = 2 // not frequent because support count < minimum support count so ignore {A, C} = 3 // not closed due to {A, C, D} {A, D} = 3 // not closed due to {A, C, D} {B, C} = 3 // not closed due to {B, C, D} {B, D} = 4 // closed but not maximal due to {B, C, D} {C, D} = 4 // closed but not maximal due to {B, C, D}

3-frequent: {A, B, C} = 2 // ignore not frequent because support count < minimum support count {A, B, D} = 2 // ignore not frequent because support count < minimum support count {A, C, D} = 3 // maximal frequent {B, C, D} = 3 // maximal frequent

4-frequent: {A, B, C, D} = 2 //ignore not frequent </

Multilevel Association Rule in data mining

Multilevel Association Rule :

Association rules created from mining information at different degrees of reflection are called various level or staggered association rules.

Multilevel association rules can be mined effectively utilizing idea progressions under a help certainty system. Rules at a high idea level may add to good judgment while rules at a low idea level may not be valuable consistently.

Utilizing uniform least help for all levels :

- At the point when a uniform least help edge is utilized, the pursuit system is rearranged.
- The technique is likewise straightforward, in that clients are needed to indicate just a single least help edge.
- A similar least help edge is utilized when mining at each degree of deliberation. (for example for mining from “PC” down to “PC”). Both “PC” and “PC” discovered to be incessant, while “PC” isn’t.

Approaches to multilevel association rule mining :

1. Uniform Support(Using uniform minimum support for all level)
2. Reduced Support (Using reduced minimum support at lower levels)
3. Group-based Support(Using item or group based support)

Let's discuss one by one.

1. **Uniform Support –**
   At the point when a uniform least help edge is used, the search methodology is simplified. The technique is likewise basic in that clients are needed to determine just a single least help threshold. An advancement technique can be adopted, based on the information that a progenitor is a superset of its descendant. the search keeps away from analyzing item sets containing anything that doesn't have minimum support. The uniform support approach however has some difficulties. It is unlikely that items at lower levels of abstraction will occur as frequently as those at higher levels of abstraction. If the minimum support threshold is set too high it could miss several meaningful associations occurring at low abstraction levels. This provides the motivation for the following approach.
2. **Reduce Support –**
   For mining various level relationship with diminished support, there are various elective hunt techniques as follows.
   - **Level-by-Level independence –**
     This is a full-broadness search, where no foundation information on regular item sets is utilized for pruning. Each hub is examined, regardless of whether its parent hub is discovered to be incessant.
   - **Level – cross-separating by single thing –**
     A thing at the I level is inspected if and just if its parent hub at the (I-1) level is regular .all in all, we research a more explicit relationship from a more broad one. If a hub is frequent, its kids will be examined; otherwise, its descendant is pruned from the inquiry.
   - **Level-cross separating by – K-itemset –**
     A-itemset at the I level is inspected if and just if it's For mining various level relationship with diminished support, there are various elective hunt techniques.
   - **Level-by-Level independence –**
     This is a full-broadness search, where no foundation information on regular item sets is utilized for pruning. Each hub is examined, regardless of whether its parent hub is discovered to be incessant.
   - **Level – cross-separating by single thing –**
     A thing at the 1st level is inspected if and just if its parent hub at the (I-1) the level is regular .all in all, we research a more explicit relationship from a more broad one. If a hub is frequent, its kids will be examined otherwise, its descendant is pruned from the inquiry.
   - **Level-cross separating by – K-item set –**
     A-item set at the I level is inspected if and just if its corresponding parents A item set (i-1) level is frequent.
3. **Group-based support –**
   The group-wise threshold value for support and confidence is input by the user or expert. The group is selected based on a product price or item set because often expert has insight as to which groups are more important than others.
   Example –
   For e.g. Experts are interested in purchase patterns of laptops or clothes in the non and electronic category. Therefore low support threshold is set for this group to give attention to these items' purchase patterns.

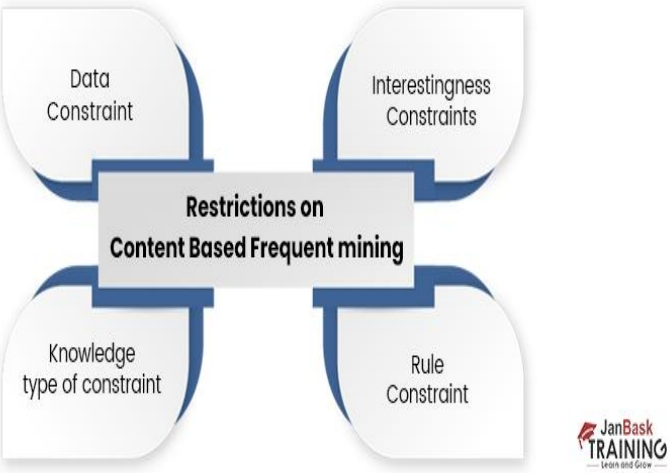
What is Constraint-Based Frequent Pattern Mining?

In order to avoid independent or tedious rules, users often want to guide data mining toward interesting patterns that align with their desired form. Constraint-based frequent mining in data mining is an approach that filters results by using predefined constraints to identify frequent patterns in large datasets. Algorithms like Apriori, FP-Growth, and Eclat are used to discover significant patterns meeting constraints such as minimum support or confidence thresholds.This helps efficiently comprehend complex data, leading to informed marketing, healthcare, and financial decision-making. Understanding constraint-based frequent pattern in data mining begins with understanding data science; you can get an insight into the same through our Data Science Training.

Constraint-Based Frequent Pattern Mining

Data mining allows for extracting thousands of rules that appear to be important from a dataset; nevertheless, likely, the majority of these rules will not provide customers with any value. Users often have a clear concept of the "*form*" of the patterns or rules they want to discover and the "*direction*" of mining that may lead to intriguing pattern discoveries. It's also possible that they have a preconceived understanding of what the "*conditions*" of the rules are, which would prohibit them from seeking rules that they already know are irrelevant to the situation. As a result of this, a useful heuristic is to have users pick constraints based on their own intuition or preconceptions about what should be allowed. This approach is referred to as "*constraint-based mining*," which is an industry term. The following are some illustrative examples of potential restrictions:

- **Knowledge Type of Constraint:** These characteristics, which may include association, correlation, categorization, or grouping, describe the nature of the knowledge that is to be mined.
- **Data Constraint:** These characteristics are used to determine the information that is required to finish a job. These constraint attempts can be guided in the right direction by imposing constraints, such as limitations on the dimensions or layers of the data, abstractions, or thought hierarchies.
- **Interestingness Constraints:** Limitations on interestingness are utilized in the process of establishing minimum and maximum values for statistica measures of rule interestingness, including support, confidence, and correlation. Limitations are placed on the interestingness of rules.
- **Rule Constraint:** The form or needs of the rules are outlined by the constraints that are placed on the rules to be mined.



In the form of metarules, one can express limitations placed on the number of predicates permitted to occur in the antecedent or consequent of a rule, as well as the relationships between attributes, attribute values, and/or aggregates (rule templates).It is necessary to have both a graphical user

interface and a high-level declarative data mining query language to be able to express such constraints.

The first four kinds of limitations have each received a substantial amount of attention over the entirety of this book and in the chapters that came before this one. In this section, we will discuss how the application of rule limits may assist in reducing the overall scope of the mining process. This constraint-based mining approach optimizes the data mining process by allowing users to describe the rules they want to uncover and then look for those rules. In addition, the user-specified limits can be utilized by an intelligent mining query optimizer, which in turn increases the mining operation's efficiency.

When using constraints, it's possible to do interactive exploratory mining and analysis. In this course, you'll learn about metarule-guided mining, a technique in which syntactic rule restrictions are described using rule templates. details the use of data space pruning (removing portions of the data space for which further exploration cannot contribute to finding patterns matching the requirements) and pattern space pruning (removing portions of the pattern space that are not being mined).

We present anti monotonicity , monotonicity, and succinctness as classes of traits that aid in pruning pattern spaces via constraint-based search space reduction. Convertible constraints are discussed; they are a subset of monotonic and anti-monotonic constraints that may be pushed farther into the iterative mining process without losing their pruning power with the right data ordering.We investigate how data space pruning might be included in a data mining workflow by introducing two sets of properties: data succinctness and data anti monotonicity .

We will assume the user is looking for association rules for the sake of discussion. Adding a correlation measure of interestingness to the support-confidence framework makes it simple to apply the proposed methods to mining correlation rules.

Metarule-Guided Mining of Association Rules

The user can specify the syntactic form of rules that the user is interested in mining when using a metarule. The mining process's efficiency may be helped by utilizing the rule forms as limitations, which can be done. Metarules can be developed manually by the analyst based on their prior knowledge, expectations, or intuition in relation to the data, or they can be generated automatically depending on the schema of the database.

EXAMPLE : Extracting with a Meteorite as a Guide

Assume you are a market analyst for AllElectronics and have access to both a list of customer transactions and demographic information about the company's clientele (such as age, residence, and credit score). You want to learn if there is a correlation between certain consumer characteristics and the products they buy. You want to know which combinations of client characteristics boost the sale of office software rather than discovering all the association rules expressing these associations. You may use a metarule to define the type of rules you want to uncover.

Metarules like **P1(X, Y) ∧ P2(X, W) ⇒ buys(X, "office software")** are examples of such expressions, where P1 and P2 are predicate variables that are instantiated to attributes from the given database during mining, X is a variable representing a customer, and Y and W take on values of the attributes assigned to P1 and P2 during the process. To examine P1 and P2 instantiations, a user will often provide a set of properties. Otherwise, a predetermined set may be applied.

In most cases, a metarule usually generates a working hypothesis about the relationships the user wishes to test or verify. As a result, the data mining software may look for a set of guidelines that correspond to the provided metarule. For example, the Rule is in agreement with the Metarule expression:

age(X, "30..39") ∧ income(X, "41K..60K")⇒buys(X, "office software").

Let's pretend we're interested in mining association rules across dimensions, like in-

Example 1. Specifically, a Metarule is a Rule Template with The Form P1 ∧ P2 ∧ ··· ∧ Pl ⇒ Q1 ∧ Q2 ∧ ··· ∧ Qr,

Where Pi (i = 1,..., l) and Qj (j = 1,..., rare instantiated predicates or predicate variables. Let p = l + r represent the total number of predicates in the metarule. Finding all common p-predicate sets, Lp, is necessary for discovering inter-dimensional association rules that fit the template.

In order to calculate the confidence of rules derived from Lp, we additionally need the support or count of the l-predicate subsets of Lp.

In this situation, multidimensional association rule mining is often applied. It is possible to construct effective strategies for metarule-guided mining by extending these approaches with the constraint-pushing techniques discussed below.

1) Reducing The Size of The Pattern Space and The Data Space in Order to Generate Constraint-Based Patterns

Rule constraints can take many different forms. Some examples of rule constraints are the projected set/subset connections of the variables in the mining rules, constant initiation of variables, and constraints on aggregate functions. Users frequently depend on their expertise with the program or data to define rule constraints for the mining work. This is done in order to prevent the mining task from becoming too complex. You can use a more conventional mining technique known as a metarule, or you may use these rule constraints instead of a metarule. In this piece, we take a look at the possible benefits in productivity that might result from incorporating rule limits into your mining operation. To begin, we will look at a case study of hybrid-dimensional association rule mining, which employs rule limitations.

During the mining process, an effective frequent pattern mining processor can narrow its focus by eliminating irrelevant patterns or discarding irrelevant data. The former determines whether or not a pattern may be trimmed by analyzing potential patterns. The Apriori attribute is used to eliminate a pattern if it cannot be further refined by mining into a super pattern . The latter examines the dataset to see if the specific data item may help generate satisfiable patterns (for a certain pattern) in the remaining mining phase. In that case, the information is omitted from further analysis. The term "*pattern pruning constraint*" refers to a constraint that can be used for pruning in the pattern space. In contrast, "*data pruning constraint*" refers to a constraint that can be used for pruning in the data space.

2) Pattern Space Pruning Through The Use of Pattern Pruning Constraints

There are five kinds of pattern mining restrictions, and each one is based on how a constraint may interact with the process of pattern mining.

1. Antimonotonic
2. Monotonic
3. Concise
4. Convertible
5. Inconvertible

3) Data Pruning: Reducing Data Volume Constraints for Pruning

In constraint-based frequent pattern mining, pruning data space is the second method of search space reduction. If a piece of data is not going to help generate satisfactory patterns in the mining process, it is removed. Shortness and non-monotonicity of data are two characteristics we take into account.

When employed at the outset of a pattern mining process to prune the data subsets that do not fulfill the constraints, constraints are considered data-succinct. Suppose a mining query specifies that the mined pattern must include the word "*digital camera*," for instance. In that case, any transactions that do not include "*digital camera*" may be removed from the dataset before mining even begins.

Intriguingly, many constraints are data anti-monotonic, meaning that they allow for the elimination of data entries that fail to conform to the present pattern during mining. Due to the fact that it cannot aid in the creation of a super pattern of the present pattern throughout the remaining mining operation, we prune it.

Advantages of Constraint-Based Mining

There are several benefits associated with using constraint-based methods for frequent pattern discovery:

- I**ncreased Efficiency** - By incorporating user-defined constraints into the algorithm design process, it's possible to reduce search space significantly.
- **Improved Accuracy** - Constraints help eliminate irrelevant or redundant results leading to more accurate findings.
- **Flexibility** - Users can customize their analysis according to their needs by defining different types of constraints.
- **Better Interpretability** - The use of additional information through constraints helps make discovered patterns more interpretable and useful than those obtained using only statistical measures like support count or confidence level.